PRÁCTICAS DE IPO

Yoel Arroyo
Iván González
Ana Isabel Molina
Carmen Lacave

Internacionalización de aplicaciones WPF

En esta sesión aprenderemos…

- Fundamentos sobre la internacionalización/globalización de una aplicación WPF
- Buenas prácticas
- Flujo de traducción interno
- Clases encargadas de la internacionalización
- Ejemplo guiado con Visual Studio

Internacionalización: Fundamentos

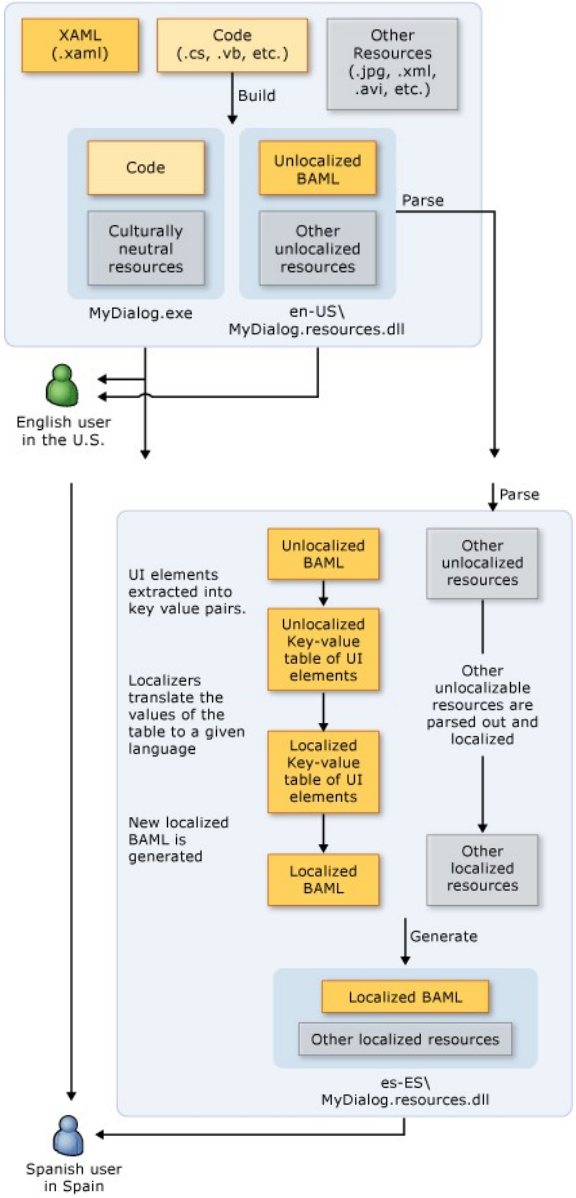
- Al limitar la disponibilidad de un producto a **un único lenguaje**, se está limitando a una pequeña fracción de clientes potenciales de la población (~ 7,5 billones).
- Para tener un **mayor alcance**, internacionalizar/globalizar el producto es una de las mejores decisiones, y a su vez, de las más económicas y efectivas.
- Internacionalizar un producto significa **adaptar una Interfaz de Usuario (IU) a distintas referencias culturales**.
- Entre las referencias culturales soportadas se puede encontrar:
  - en-US: inglés (Estados Unidos)
  - en-GB: inglés (Gran Bretaña)
  - fr-FR: francés (Francia)
  - zh-HK: chino (Hong-Kong)
  - es-ES: español (España)
- Para que una aplicación WPF sea internacional, los desarrolladores deben **generar y compilar un recurso traducible por cada idioma deseado**.

Internacionalización: Buenas Prácticas

- **Evitar** especificar la interfaz de usuario usando código **C#**.
- En su lugar, **definir** la interfaz de usuario usando **XAML**.
- **Evitar** usar **posiciones absolutas** y **tamaños fijos** sobre los elementos de la interfaz de usuario. En su lugar, utilizar redimensionado automático y relativo:
  - Evitar usar el componente "*Canvas*" para definir la distribución de los elementos de la interfaz.
  - Usar "*Grid*" y sus capacidades de redimensionado en su lugar.
- **Proporcionar tamaño extra** utilizando márgenes, puesto que el texto traducible a menudo requiere de más espacio. El espacio adicional permite añadir más caracteres al elemento.
- **Habilitar** la propiedad "*TextWrapping*" en un ***TextBlock*** para evitar que el texto traducible se recorte debido a falta de espacio suficiente.

Internacionalización: Flujo de traducción

1. El desarrollador crea y globaliza la aplicación WPF.
2. Establece una *UICulture* por defecto en las propiedades del proyecto. Por ejemplo: *inglés="en"*.
3. Genera un fichero de recursos por cada idioma que desee dar soporte.
4. De esta forma, cuando se compila la aplicación, se genera un ensamblado principal independiente del idioma.
5. Este ensamblado tiene un archivo satélite *x.resources.dll* que contiene todos los recursos traducibles.
6. Cuando el archivo se compila, XAML se convierte al formato BAML de XAML para su posterior ejecución.



Internacionalización: CurrentCultureInfo -CurrentCulture & UICurrentCulture-

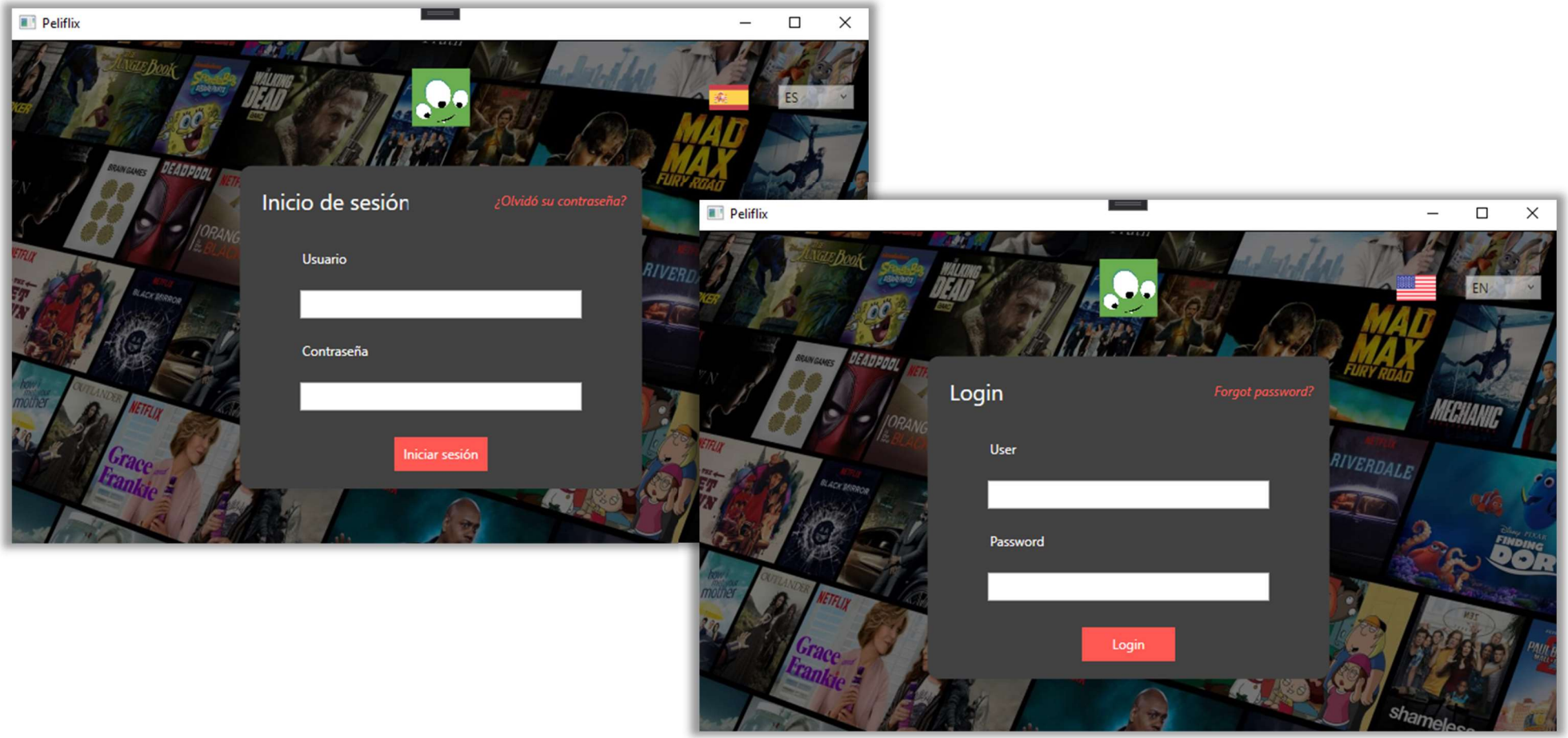
- La **clase** CultureInfo proporciona información sobre una **cultura/localización** específica, tales como su idioma, sublenguaje, país/región, calendario, y otras convenciones asociadas a una cultura en concreto.
- Esta clase también proporciona acceso a instancias de objetos de una cultura específica, tales como *DateTimeFormatInfo*, *NumberFormatInfo*, *CompareInfo*, y *TextInfo*.
- Estos objetos contienen **información requerida para operar sobre una cultura específica**, tales como transformar texto entre mayúsculas/minúsculas, formatear fechas y números, y comparar cadenas.
- La clase *CultureInfo* también permite **especificar** un **nombre único** para cada cultura específica:
  - Este nombre está formado por la combinación de un código ISO 639 formado por dos letras minúsculas, identificando el idioma, y un segundo código ISO 3166 formado por dos letras minúsculas, identificando el país/región. **Por ejemplo: Español (España) = “es-ES”; Español (México) = “es-MX”.**

Internacionalización: CurrentCultureInfo -CurrentCulture & UICurrentCulture-

- La clase CultureInfo también permite especificar **culturas neutrales**:
  - Una cultura neutral se especifica con un código formado sólo por dos letras en minúscula. Por ejemplo: Francés = “fr”; Alemán = “de”; Inglés = “en”.
  - Tan sólo dos culturas contradicen esta norma. La cultura *china (simplificada)*, cuyo código es “zh-Hans”, y *china (tradicional)*, cuyo código es “zh-Hant”.
- Todo **hilo** de ejecución en una aplicación WPF posee una “*current culture*” y una “*current UI culture*”.
- *CurrentCulture* determina la convención en el formateo de fechas, tiempos, números y precios, el orden del texto, mayúsculas/minúsculas, y en la comparación de cadenas.
- *CurrentUICulture* es utilizada para recuperar y/o modificar la cultura específica de los recursos en tiempo de ejecución.
- **Cambiar el idioma** de una aplicación WPF resulta sencillo si se modifica el valor de dichos objetos, consultando el **hilo de ejecución principal** de la aplicación WPF y manipulando el objeto *CultureInfo*.

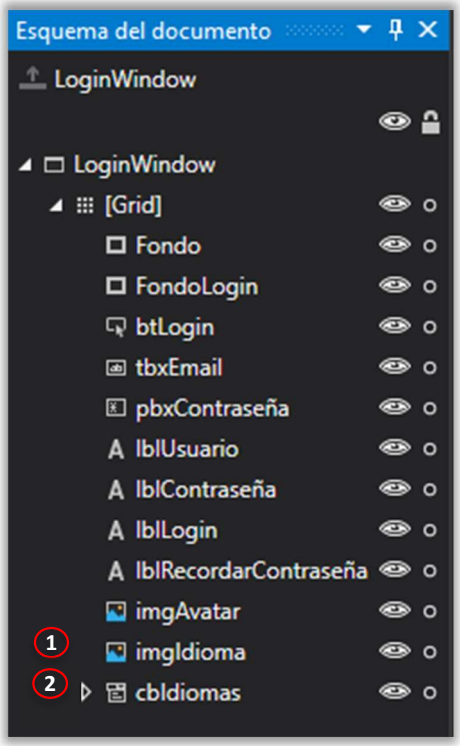
Ejemplo guiado

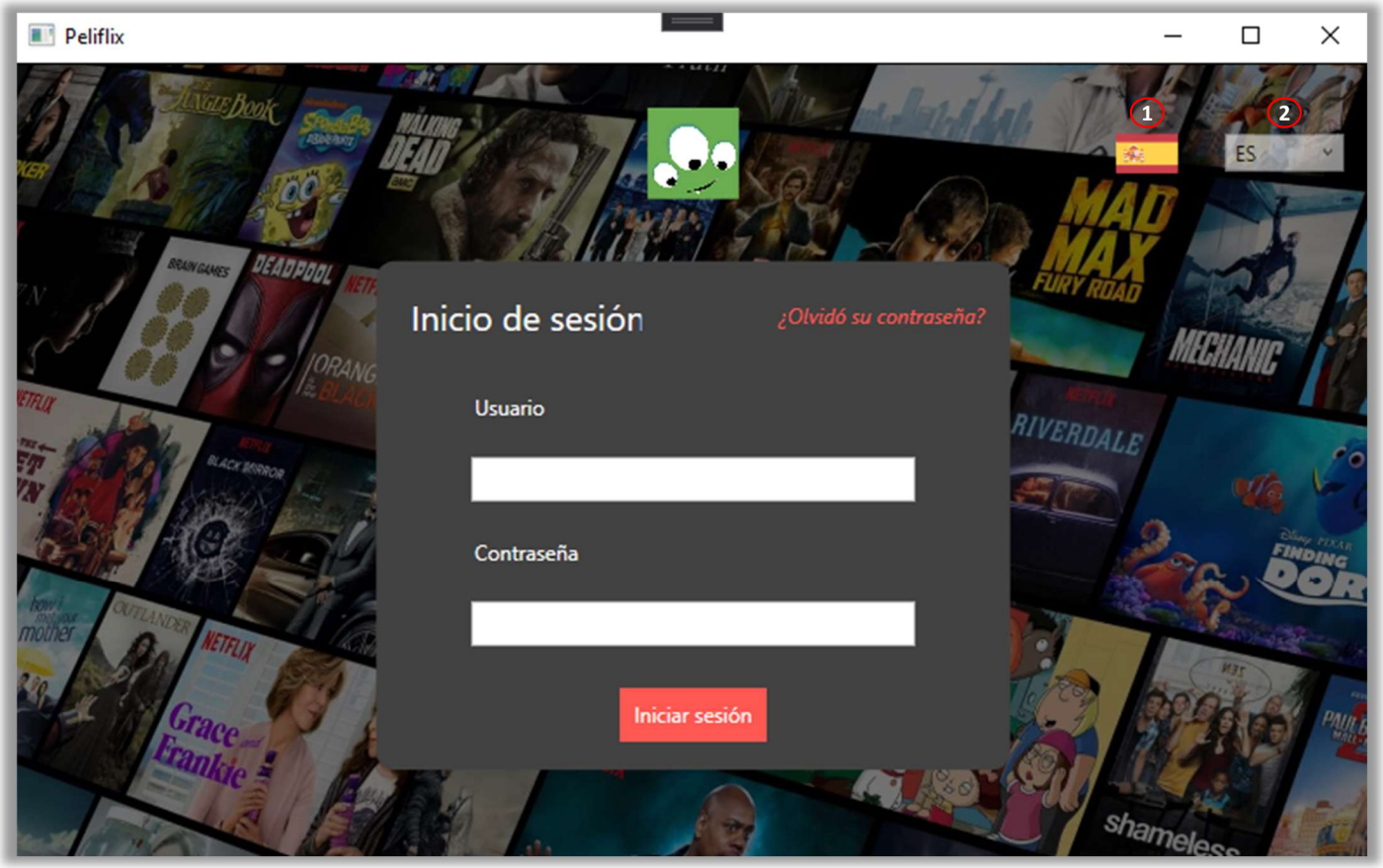
- Internacionalizar el formulario para “iniciar sesión” en Peliflix



Ejemplo guiado

- Componentes de la interfaz de usuario:

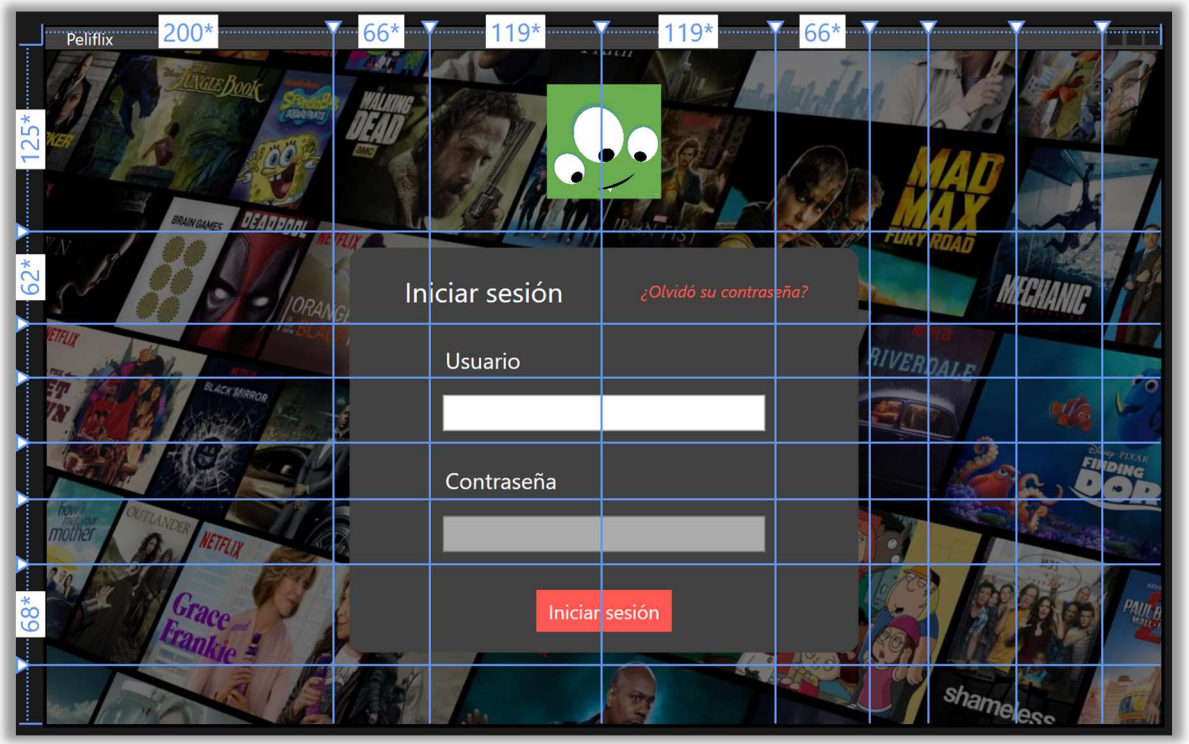




Ejemplo guiado

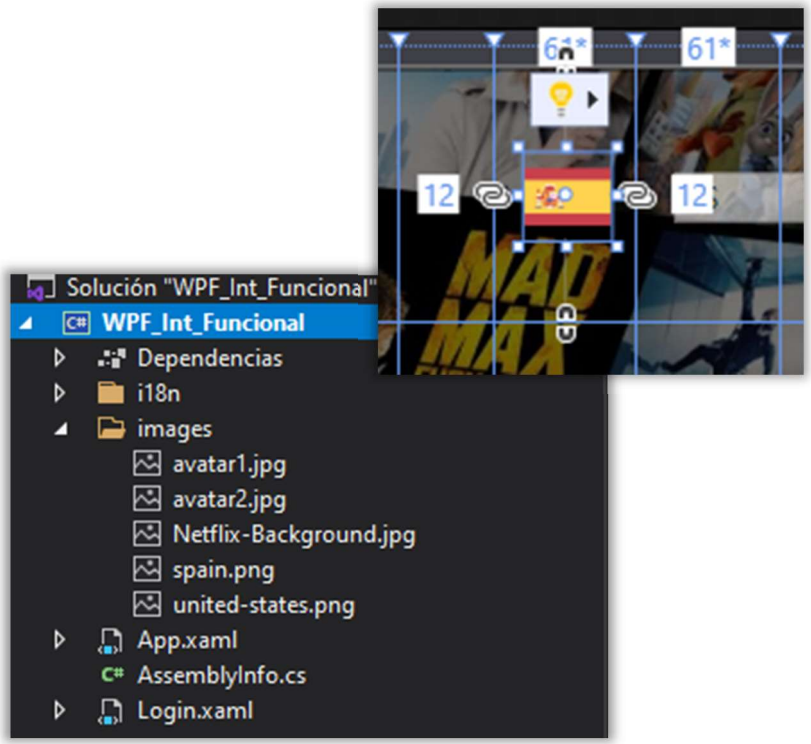
- Componentes de la interfaz de usuario:
  - Grid:

```
<Grid.RowDefinitions>
      <RowDefinition Height="125*"/>
      <RowDefinition Height="62*"/>
      <RowDefinition Height="38*"/>
      <RowDefinition Height="44*"/>
      <RowDefinition Height="38*"/>
      <RowDefinition Height="44*"/>
      <RowDefinition Height="68*"/>
      <RowDefinition Height="40*"/>
</Grid.RowDefinitions>
<Grid.ColumnDefinitions>
      <ColumnDefinition Width="200*"/>
      <ColumnDefinition Width="66*"/>
      <ColumnDefinition Width="119*"/>
      <ColumnDefinition Width="119*"/>
      <ColumnDefinition Width="66*"/>
      <ColumnDefinition Width="40*"/>
      <ColumnDefinition Width="60*"/>
      <ColumnDefinition Width="60*"/>
      <ColumnDefinition Width="40*"/>
</Grid.ColumnDefinitions>
```



Ejemplo guiado

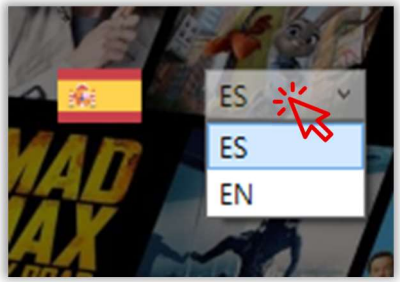
- Componentes de la interfaz de usuario:
  - imgIdioma (Image):
    - Añadir nuevas imágenes con las banderas de España y EE.UU en carpeta “images” *




```
<Image x:Name="imgIdioma"
    Grid.Column="6"
    HorizontalAlignment="Right"
    Margin="12,0,0,0"
    VerticalAlignment="Center"
    Source="/images/spain.png"
    Width="42" Height="42"/>
```

Ejemplo guiado

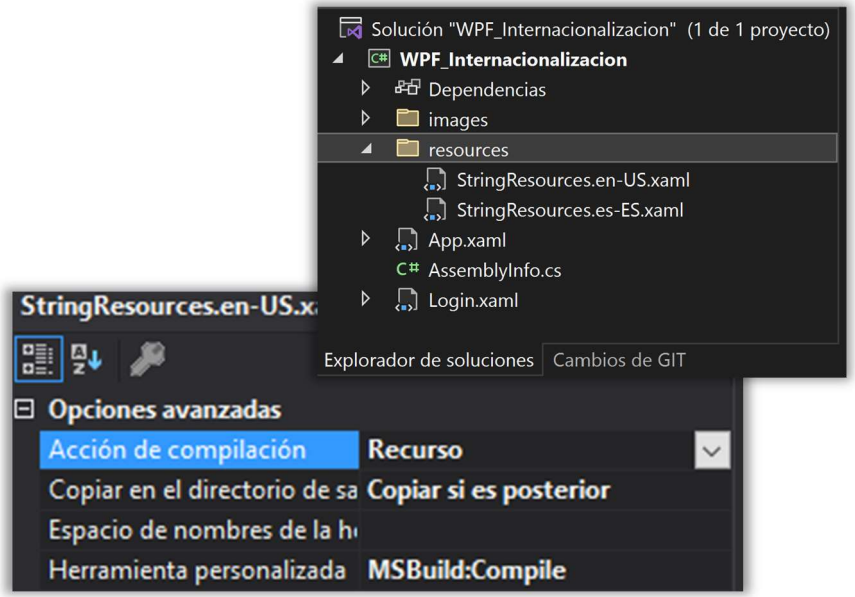
- Componentes de la interfaz de usuario:
  - cbIdiomas (ComboBox):

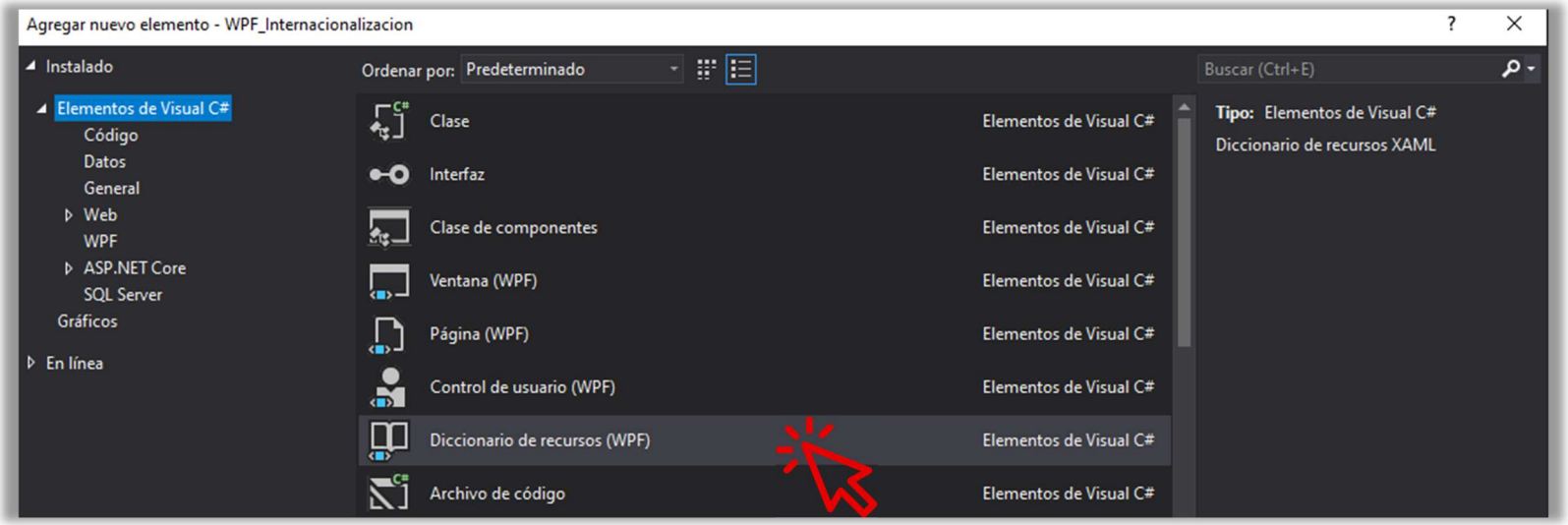



```
<ComboBox x:Name="cbIdiomas"
    Grid.Column="7" HorizontalAlignment="Left"
    Margin="5,0,35,0" VerticalAlignment="Center"
    Grid.ColumnSpan="2" Opacity="0.6"
    SelectedIndex="0”
    SelectionChanged="cb_elementoSeleccionado"
    Background="Black" Height="22" Padding="5,3,5,3"
    MaxWidth="80" Width="52">
    <ComboBoxItem x:Name="cbItem1"
        Content="ES"/>
    <ComboBoxItem x:Name="cbItem2"
        Content="EN"/>
</ComboBox>
```

Ejemplo guiado

- Dotando de funcionalidad:
  - Agregar los diccionarios de recursos (I):
    - Archivo “StringResources.**en-US**.xaml” para idioma inglés americano (United States)
    - Archivo “StringResources.**es-ES**.xaml” para idioma español de España
    - Los diccionarios de recursos deben contener el mismo nombre, añadiendo la *localización* del idioma deseado





Ejemplo guiado

- Dotando de funcionalidad:
  - Agregar los diccionarios de recursos (II):
    - Añadir los literales para cada uno de los archivos en su idioma correspondiente
    - Es importante que el nombre de la clave sea idéntico en todos los archivos
    - Los ficheros de recursos deben contener el mismo nombre, añadiendo la *localización* del idioma deseado

```xml
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
                    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
                    xmlns:sys="clr-namespace:System;assembly=mscorlib">

    <sys:String x:Key="WindowTitle">Movieflix</sys:String>
    <sys:String x:Key="btLogin">Login</sys:String>
    <sys:String x:Key="cbItem1">SP</sys:String>
    <sys:String x:Key="lblContraseña">Password</sys:String>
    <sys:String x:Key="lblLogin">Login</sys:String>
    <sys:String x:Key="lblRecordarContraseña">Forgot password?</sys:String>
    <sys:String x:Key="lblUsuario">User</sys:String>

</ResourceDictionary>
```

StringResources.en-US.xaml

Ejemplo guiado

- Dotando de funcionalidad:
  - Agregar los diccionarios de recursos (II):
    - Añadir los literales para cada uno de los archivos en su idioma correspondiente
    - Es importante que el nombre de la clave sea idéntico en todos los archivos
    - Los ficheros de recursos deben contener el mismo nombre, añadiendo la *localización* del idioma deseado

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
                    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
                    xmlns:sys="clr-namespace:System;assembly=mscorlib">

    <sys:String x:Key="WindowTitle">Peliflix</sys:String>
    <sys:String x:Key="btLogin">Iniciar sesión</sys:String>
    <sys:String x:Key="cbItem1">ES</sys:String>
    <sys:String x:Key="lblContraseña">Contraseña</sys:String>
    <sys:String x:Key="lblLogin">Inicio sesión</sys:String>
    <sys:String x:Key="lblRecordarContraseña">¿Olvidó su contraseña?</sys:String>
    <sys:String x:Key="lblUsuario">Usuario</sys:String>

</ResourceDictionary>
```

StringResources.es-ES.xaml

Ejemplo guiado

- Dotando de funcionalidad:
  - Agregar los archivos de recursos (III):
    - Asociar los diccionarios de recursos a la Aplicación
    - Editar el fichero **App.xaml**

```xml
<Application x:Class="WPF_Int_Funcional.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             xmlns:local="clr-namespace:WPF_Int_Funcional"
             StartupUri="Login.xaml">
    <Application.Resources>
        <ResourceDictionary>
            <ResourceDictionary.MergedDictionaries>
                <ResourceDictionary Source="resources/StringResources.en-US.xaml" />
                <ResourceDictionary Source="resources/StringResources.es-ES.xaml" />
            </ResourceDictionary.MergedDictionaries>
        </ResourceDictionary>
    </Application.Resources>
</Application>
```

Ejemplo guiado

- Dotando de funcionalidad:
  - Funcionalidad para el cambio de idioma:
    - Añadir método para control de cambio de idioma
    - Editar el fichero **App.xaml.cs**

App.xaml.cs

```
public static ResourceDictionary SelectCulture(string idioma) {
            var resourceDictionary = new ResourceDictionary();
            switch (idioma) {
                case "en-US":
                    resourceDictionary.Source =
                          new Uri("/resources/StringResources.en-US.xaml", UriKind.Relative);
                    break;
                case "es-ES":
                    resourceDictionary.Source =
                          new Uri("/resources/StringResources.es-ES.xaml", UriKind.Relative);
                    break;
                default:
                    resourceDictionary.Source =
                          new Uri("/resources/StringResources.es-ES.xaml", UriKind.Relative);
                    break;
            }

            Thread.CurrentThread.CurrentCulture = new CultureInfo(idioma);
            Thread.CurrentThread.CurrentUICulture = new CultureInfo(idioma);

            return resourceDictionary;
        }
```

Ejemplo guiado

- Dotando de funcionalidad:
  - Asociar los literales a los elementos de la interfaz de usuario:
    - Asociar a cada elemento su clave correspondiente
    - Utilizar el fichero XAML
    - Nomenclatura: {DynamicResource Key}
    - Ejemplo para etiqueta “lblUsuario”: {DynamicResource lblUsuario}

```
Title="{DynamicResource WindowTitle}“ Height="500" Width="800"
   WindowStyle="SingleBorderWindow" ResizeMode="CanResize"
   WindowStartupLocation="CenterScreen" MaxWidth="1024"
   MaxHeight="800">
```

```
<Label x:Name="lblUsuario"
   Content="{DynamicResource lblUsuario}"
   Foreground="White" RenderTransformOrigin="0.902,-0.345"
   Grid.Column="2" Grid.Row="2“
   VerticalContentAlignment="Center"
   Margin="5,5,10,5" Grid.ColumnSpan="2"/>
```

Ejemplo guiado

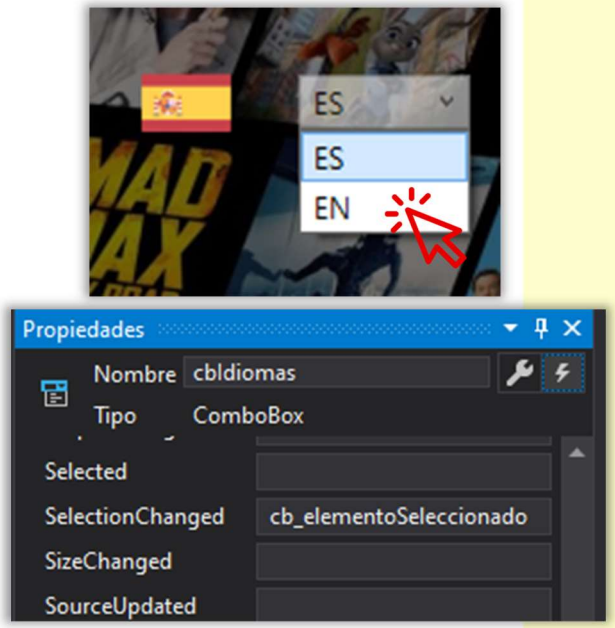
- Dotando de funcionalidad:
  - Implementar lógica para el cambio de idioma (estático –al desplegar una ventana-):

```
public Login()
     {
          InitializeComponent();
          Resources.MergedDictionaries.Add(App.SelectCulture("es-ES"));
          cambiarBandera("es-ES");
     }

private void cambiarBandera(string idioma)
     {
          imgIdioma.Source = idioma.Equals("en-US")
          ? new BitmapImage(new Uri("/images/united-states.png",
               UriKind.Relative))
               : new BitmapImage(new Uri("/images/spain.png",
                       UriKind.Relative));
     }
```

Ejemplo guiado

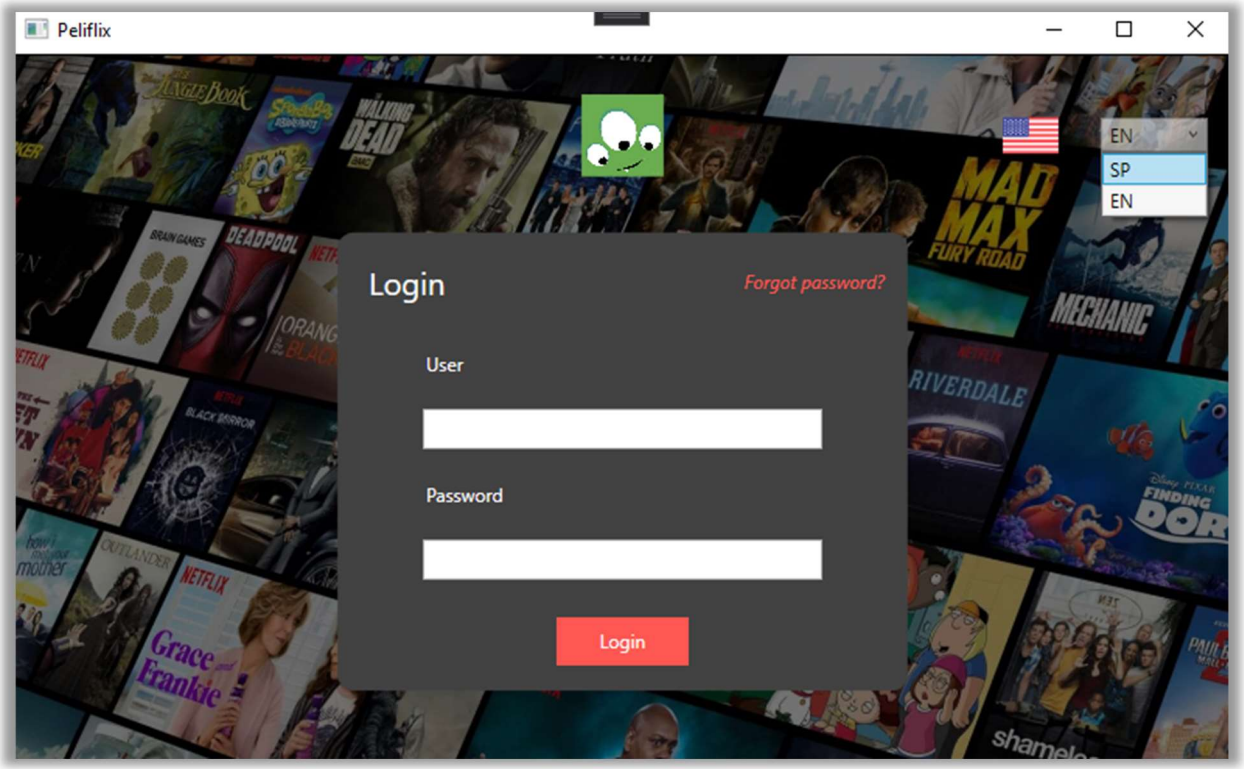
- Dotando de funcionalidad:
  - Implementar lógica para el cambio de idioma (dinámico –provocado por un evento-):




```
private void cb_elementoSeleccionado(object sender, SelectionChangedEventArgs e)
      {
            ComboBoxItem cbi = (ComboBoxItem)cbIdiomas.SelectedItem;
            string selectedText = cbi.Content.ToString();

            if ((selectedText.Equals("ES") || selectedText.Equals("SP"))
                && !CultureInfo.CurrentCulture.Name.Equals("es-ES"))
            {
                Resources.MergedDictionaries.Add(App.SelectCulture("es-ES"));
                cambiarBandera("es-ES");
            }
            else if (selectedText.Equals("EN")
                && !CultureInfo.CurrentCulture.Name.Equals("en-US"))
            {
                Resources.MergedDictionaries.Add(App.SelectCulture("en-US"));
                cambiarBandera("en-US");
            }
      }
```

Resultado. Propuesta de mejora



- Agregar soporte para el idioma francés
- Generar una nueva ventana con soporte español, inglés y francés al pulsar en “¿Olvidó su contraseña?”

WPF
Microsoft .NET
Windows Presentation Foundation